

Міністерство освіти та науки України

Національний університет “Львівська політехніка”

Інститут інформаційно-комунікаційних технологій та електронної
інженерії



ЗВІТ

З Лабораторної роботи №6
з дисципліни “Об’єктно-орієнтовне програмування частина 2”

Виконав:

Ст. гр. ІХ-22

Ненадовський Р.М.

Прийняв:

Плесканка М.В.

Ліviv 2026

Тема: Асинхронне програмування в телекомунікаційних системах: взаємодія з базою даних через SQLAlchemy.

Мета: Ознайомитися з принципами асинхронності в Python (asyncio) та навчитися працювати з асинхронними запитам до БД через SQLAlchemy ORM.

Завдання:

Реалізувати клієнт-серверну взаємодію через REST API:

- створити клієнт для відправки запитів;
- забезпечити обробку відповідей;
- використовувати стандартні HTTP-методи.

Код:

```
import requests

class RestClient:
    def __init__(self, base_url):
        """Ініціалізація базової URL-адреси для API [cite: 39, 40]"""
        self.base_url = base_url

    def get(self, endpoint):
        """Виконує HTTP GET-запит та повертає дані [cite: 119]"""
        url = f"{self.base_url}{endpoint}"
        try:
            response = requests.get(url)
            # Перевірка статус-коду (200 OK) [cite: 49, 125]
            if response.status_code == 200:
                return response.json()
            else:
                return f"Помилка {response.status_code}: {response.text}"
        except Exception as e:
            return f"Виникла помилка при GET-запиті: {e}"

    def post(self, endpoint, data):
        """Виконує HTTP POST-запит з переданими даними [cite: 120]"""
        url = f"{self.base_url}{endpoint}"
        try:
            response = requests.post(url, json=data)
            # Перевірка статус-коду (201 Created) [cite: 71, 78]
            if response.status_code == 201:
                return response.json()
            else:
                return f"Помилка {response.status_code}: {response.text}"
        except Exception as e:
            return f"Виникла помилка при POST-запиті: {e}"
```

```
# --- Демонстрація роботи ---

if __name__ == "__main__":
    # Створюємо екземпляр клієнта для JSONPlaceholder [cite: 40]
    client = RestClient("https://jsonplaceholder.typicode.com/")

    print("--- Тестування GET-запиту (отримання списку постів) ---")
    posts = client.get("posts")
    if isinstance(posts, list):
        print(f"Отримано {len(posts)} постів. Перші два:")
        print(posts[:2]) # Виводимо перші 2 записи [cite: 51]

    print("\n--- Тестування POST-запиту (створення нового посту) ---")
    new_post_data = {
        "title": "Мій новий пост",
        "body": "Це текст для лабораторної роботи",
        "userId": 1
    }
    created_post = client.post("posts", new_post_data)
    print("Сервер повернув створений об'єкт:")
    print(created_post) # Ось тут видали [cite: 72]
```

Результат:

```
--- Тестування GET-запиту (отримання списку постів) ---
Отримано 100 постів. Перші два:
[{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio ne
|
--- Тестування POST-запиту (створення нового посту) ---
Сервер повернув створений об'єкт:
{'title': 'Мій новий пост', 'body': 'Це текст для лабораторної роботи', 'userId': 1, 'id': 101}

Process finished with exit code 0
```

Висновок: Застосування асинхронного підходу дозволяє одночасно обробляти багато мережевих вузлів без блокування програми. Використання `AsyncSession` у `SQLAlchemy` забезпечує ефективне збереження та оновлення статусів об'єктів у базі даних.